AMITY SCHOOL OF ENGINEERING & TECHNOLOGY

AMITY UNIVERSITY MUMBAI

AMITY UNIVERSITY MUMBAI

(Academic Year 2026- 27)

INDEX

Sr. No.	Description	Date	Page No.	Grade
1	LPP to maximize profit in linear optimization.	12/02	1-3	
2	Checking saddle point in a game using pure strategy.	26/02	4-7	
3	Generate payoff matrix for 2 player game.	05/03	8-11	
4	Implement game using mixed strategy.	02/04	12-15	
5	Solving optimization problem using array.	09/04	16-18	
6	Travelling salesman problem implementation using integer programming.	4/04	19-22	
7	Implementing Ford-Fulkerson algorithm.	16/04	23-26	
8	Implementing minimum cost flow algorithm.	23/04	27-29	
9	Solving employee scheduling Problem.	30/04	30-32	
10	Implementing assignment problem in LPP.	07/05	33-35	
11	using Branch and Bound method to calculate Knapsack.	09/05	36-39	

EXPERIMENT NO. 01

* **AIM :-** Implementation of linear programming optimization problem to maximize profit.

* **THEORY :-** Linear programming problem is a mathematical technique which is used to determine best decision variables that produce the highest possible profit while satisfying a constraints. The objective function represent profit. Constraints represents limitation like budget, labor cost, etc. In linear programming, both objective function and constraints are linear equations or inequalities Feasible region contains all possible solutions like satisfying constraint, and optimal solution lies in a boundary point.

* **CODE :-**

```
import numpy as np
from scipy.optimize import linprog

# coefficients for objective function (-ve for maximization)
C = [-80, -90]

# Inequality Constraints (A_ub * x <= b_ub)
A = [(2, 1),
     (1, 1)]

B = [100, 80]
```

```
# Bound for x and y
x_bound = (0, None)
y_bound = (0, None)

# Solve
result = linprog(c, A_ub=A, b_ub=b, bounds=[x_bounds, y_
         bounds])

# Print results
print("Optimal production:", result.x)
print("Maximum profit:", -result.fun)
```

PROBLEM USED :-

Maximize $P = -80x - 90y$

Subject to,

$$2x + y \leq 100$$
$$x + y \leq 80$$
$$x, y \geq 0$$

* END *

* OUTPUT :-

Optimal production : [20, 80]

Maximum profit : £7,200

Page No. 01
Date 10 04 2026

EXPERIMENT NO.02

AIM :- Write code to check saddle point exists in a game using pure strategy

THEORY :- A saddle point in a game theory exists when the maximum value equals the minimax value in payoff matrix. The maximin is obtained by finding the minimum value in each row and then selecting maximum among them. The minimax is found by taking maximum value in each column and choosing the minimum among them. If both values are equal, saddle point exists in the game.

CODE :-

```
# Function to check saddle point
def saddle_point (matrix, r, c):
    # Find row minima
    row_min = []
    for i in range (r):
        row_min. append (min (matrix [i]))
maximin = max (row_min
# Find column maxima
col_max = []
for j in range (c)
    col = []
    for i in range (r):
        col. append (matrix [i][j]
    Col_max. append (max (col))
minimax = min (col_max)
# Check condition
```

```
if maximin == minimax :
    print ("Saddle point exists")
    print ("Value of game:", maximin)
else:
    print ("No Saddle point exists")

#----- Input -----
r = int (input ("Enter number of rows:"))
c = int (input ("Enter number of columns:"))
matrix = []
print ("Enter matrix elements:")
for i in range (r):
    row = list (map (int, input (). split ()))
    matrix. append (row)

#----- function call ------
Saddle point (matrix, r, c)
```

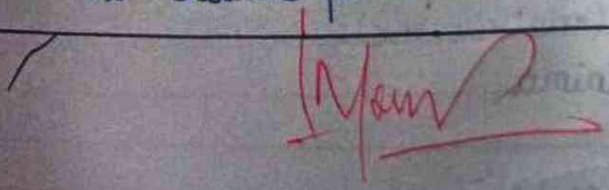
* **CONCLUSION :-**

Checking of saddle point existence in a game using pure strategy is successfully implemented. No saddle point exists.

* ! END ! *

* OUTPUT :-

```
Enter number of rows : 2
Enter number of columns : 2
Enter matrix elements :
3 1
0 2
No Saddle point exists.
```



EXPERIMENT NO.03 01

* AIM :- Write a code to generate payoff matrix for a 2 player game.

* THEORY :- A payoff matrix in a two player game is outcomes of different strategy combination between player A and player B. Each cell shows payoff recieved by players for particular strategies. Player A chooses row strategy while Player B chooses column strategy. The matrix helps analyse competitive situation and determine optimal strategies. It is widely used in zero sum games. By studying it, we can find the saddle points, dominant strategies, etc.

* CODE :-

```
# Input number of strategies
rows = int (input ("Enter no of strategy for A:"))
cols = int (input ("Enter no. of strategy for B:"))
# Create empty matrix
matrix = []
print ("\n Enter payoff values:")
# Input values manually
for i in range (rows):
    row = []
    for j in range (cols):
        val = int (input (f"Enter value for A(i+1), B(j+1): "))
        row.append (val)
```

```
matrix.append(row)
# Display matrix
print("\n payoff matrix:\n")

# Column headers
print("  ", end="")
for j in range(cols):
    print(f"B{j+1}", end="  ")
print()

# Rows with values
for i in range(rows):
    print(f"A{i+1}", end="  ")
    for j in range(cols):
        print(f"{matrix[i][j]}", end="  ")
    print()
```

* CONCLUSION :-

The payoff matrix for 2 player game is successfully generated showcasing both player A (rows) and player B (columns) strategies.

* ! END ! *

* OUTPUT :-

Enter no. of strategy for player A : 2

Enter no. of strategy for player B : 2

Enter payoff values :

Enter value for A_1, B_1 : 9

Enter value for A_1, B_2 : 4

Enter value for A_2, B_1 : 8

Enter value for A_2, B_2 : 5

Payoff matrix :

	B_1	B_2
A_1	9	4
A_2	8	5

EXPERIMENT NO.04

* **AIM :-** Write a code to implement a game by using mixed strategy.

* **THEORY :-** A game is solved using mixed strategy when no saddle point exists. In this approach, each player assigns probabilities to their strategies instead of choosing a single pure strategy. The objective is to make the opponent indifferent between their choices. For a 2×2 game, we assume probabilities p and (1-p) for Player A and q and (1-q) for Player B. We equate payoff of strategies to find optimal probabilities. After solving equations, we obtain equilibrium mixed strategies and the value of the game. Method ensures optimal play under uncertainty and guarantees a stable solution in a non-pure strategy game or the mixed strategy game.

* **CODE :-**

```
# 2x2 Mixed strategy Game Solver
a, b = map (int, input ("Enter row 1:"). split ())
c, d = map (int, input ("Enter row 2:"). split ())

D= (a+b) - (b+c)
```

```
if D == 0
    print ("No unique solution")
else:
    p = (d - c) / D
    q = (d - b) / D
    v = (a * b - d * c) / D

    print ("Player A: ", round (p, 2), round (1-p, 2))
    print ("Player B: ", round (q, 2), round (1-p, 2))
    print ("Value : ", round (v, 2))
```

* **CONCLUSION :-**

Game is successfully implemented using mixed strategy.

* ! END ! *

* OUTPUT :-

```
Enter row : 4  7
Enter row : 8  9
Player A : -0.5  1.5
Player B : -1.0  2.0
Value : 10
```

EXPERIMENT NO. 01

* **Aim :-** To define and solve optimization model using arrays with numpy and scipy.optimize.linprog.

* **THEORY :-** Optimization model represent real-world decision problem mathematically. Using arrays to define LP models is efficient, scalable and readable. Numpy array provide efficient matrix operations, which are internally used by scipy linprog solver. Array-based model definition make it easy to scale upto large problems with many variables and the constraints.

* **CODE :-**

```
import numpy as np
from scipy.optimize import linprog
# coefficient of objective fn. (max → convert to minimize)
C = np.array([-3, -5])
# Constraint coefficients
A = np.array([[2,3]
              [1,1]])
# Right-hand side
b = np.array([12,6])

# Bounds for variables (x1, x2 >= 0)
x_bounds = (0, None)
y_bounds = (0, None)

#Solve
```

```
result_ = linprog(c, A_ub=A, b_ub=b, bounds=(x_
                  bounds_, y_bounds_))

print('Optimal value:', -result.fun)
print('x1, x2':, result.x)
```

* CONCLUSION :-

The Optimization model was successfully defined using arrays and solved with scipy linprog. The result $x_1 = 0$, $x_2 = 4.0$ with optimal value 20.0 confirm that array-based LP formulation is the concise value.

* ! END ! *

* OUTPUT :-

Optimal value = 20.0

x_1, x_2 : [0, 4]

Status : Optimization terminated successfully

(HIGHS status 7 : Optimal)

EXPERIMENT NO. 01

* AIM :- Travelling salesman problem using Integer linear programming (ILP).

* THEORY :- TSP asks given set of cities and distance between them, find shortest possible tour that visit each city exactly once and return to the origin. Condition or constraint is that every city should be visited once and should not be repeated afterwards.

* CODE :-

```
import pulp
import numpy as np
# Distance matrix
dist = np.array([[0, 10, 15, 20],
                 [10, 0, 35, 25],
                 [15, 35, 0, 30],
                 [20, 25, 30, 0]])
n = len(dist)
# Create model
model = pulp.LpProblem('TSP', pulp.LpMinimize)
# Decision variables
x = pulp.LpVariable.dict('x', (range(n), range(n)), cat='Binary')
# MTZ variables
u = pulp.LpVariable.dict('u', range(n), lowBound=0, upBound=n-1, cat='Integer')
# Objective function
model += pulp.lpSum(dist[i][j] * x[i][j] for i in range
```

```
(n) for j in range(n))
# No self-loop
for i in range(n):
    model += x[i][i]==0
# Each city visited once (outgoing)
for i in range(n):
    model += pulp.lpSum(x[i][j] for j in range(n)
    ==1
# Each city visited once (incoming)
for j in range(n):
    model += pulp.lpSum(x[i][j] for i in range(n)==1
# Solve
model.solve()
# Output tour
for i in range(n):
    for j in range(n):
        if pulp.value(x[i][j]==1):
            print(f'[i]→[j]')
# MTZ subtour elimination
for i in range(1,n):
    for j in range(1,n):
        if i!=j
            model += u[i]-u[j]+n*x[i][j]<=n-1
```

* **CONCLUSION :-**

TSP is solved successfully.

* OUTPUT :-

Status : Optimal

Tour : 0 → 2 (dist : 15)

1 → 0 (dist : 10)

2 → 3 (dist : 30)

3 → 1 (dist : 25)

Total tour cost : 80.

EXPERIMENT NO. 01

* **AIM :-** Ford - Fulkerson Algorithm for maximum flow.

* **THEORY :-** Ford - Fulkerson algorithm solves maximum flow problem given a directed graph with source S and sink T, and edge capacities, finds maximum flow that can be routed from S to T.
Concepts are :
Residual graph, augmenting path, Bottleneck capacity and max-flow min-cut theorem.
Time complexity is O(VE).

* **CODE :-**

```
import networkx as nx
G = nx.DiGraph()
# Add edges with capacities
G.add_edge('S', 'A', capacity=16); G.add_edge('B', 'D', capacity=14)
G.add_edge('S', 'B', capacity=13); G.add_edge('C', 'B', capacity=9)
G.add_edge('A', 'B', capacity=10); G.add_edge('C', 'T', capacity=20)
G.add_edge('A', 'C', capacity=12); G.add_edge('D', 'C', capacity=7)
G.add_edge('B', 'A', capacity=4); G.add_edge('D', 'T', capacity=4)
flow_value, flow_dict = nx.maximum_flow(G, 'S', 'T')
print('Maximum flow:', flow_value)
```

* **CONCLUSION :-**

Ford - Fulkerson was implemented successfully.

* ! END ! *

* OUTPUT :-

Maximum flow : 23

Flow on each edge :

S → A : 13

S → B : 10

A → B : 1

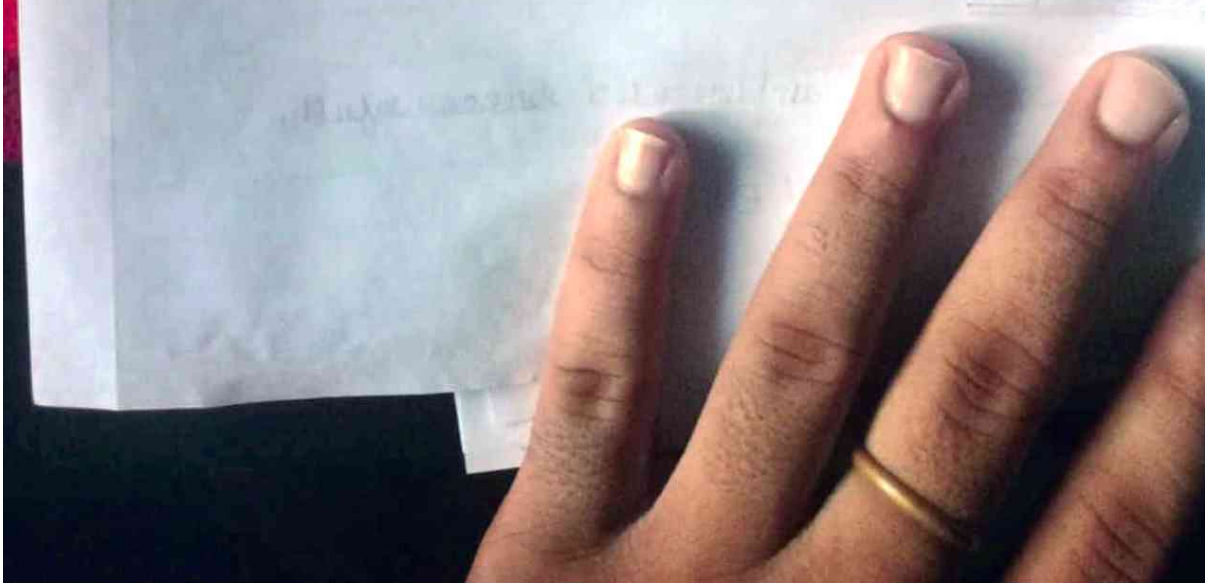
A → C : 12

B → D : 11

C → T : 19

D → C : 7

D → T : 4



EXPERIMENT NO. 01

* **AIM :-** Minimum cost flow Algorithm

* **THEORY :-** MCF problem finds cheapest way to send a required amount of flow through a network. Unlike maximum flow, MCF also consider edge cost (weights).
Application include supply chain logistic, traffic routing, and resource allocation where both flow quantity and cost must be optimized.
The Successive shortest path algorithm and network simplex method are common approaches. Networkx implements the Network Simplex algorithm (min_cost_flow).

* **CODE :-**

```
import networkx as nx
G = nx.DiGraph()
# Add edges
G.add_edge('S', 'A', capacity=10, weight=2)
G.add_edge('S', 'B', capacity=5, weight=4)
G.add_edge('A', 'B', capacity=15, weight=1)
G.add_edge('A', 'T', capacity=10, weight=2)
G.add_edge('B', 'T', capacity=10, weight=3)

# Demand (flow requirement)
G.nodes['S']['demand'] = -15  # Supply
G.nodes['T']['demand'] = 15  # demand
```

```
# Solve
flow_dict = nx.min_cost_flow(G)
cost = nx.cost_of_flow(G, flow_dict)

print('Flow', flow_dict)
print('Minimum cost:', cost)
```

* **CONCLUSION :-**

The MCF algorithm routed 15 units from S to T at minimum total cost of 75. The Solver chose direct paths S→A→T (10 units, cost 20) and S→B→T (5 units, cost 35), bypassing the cheaper A→B intermediate edge to satisfy flow conservation at minimum cost.

* ! END ! *

* OUTPUT :-

Flow on each edge :

S → A : 10 units

S → B : 5 units

A → T : 10 units

B → T : 5 units

Minimum Cost : 75

EXPERIMENT NO : 01

* **AIM :-** Employee scheduling Problem using Linear programming.

* **THEORY :-** The Employee scheduling Problem assigns employee to shift to meet coverage requirement at minimum cost. It is a classic combinational optimization problem.
Binary variable enforce that each assignment is either (1) or not (0). PuLP uses branch-and-bound internally to solve the ILP problems.

* **CODE :-**

```
import pulp
# Employee and shifts
employee = ['E1', 'E2', 'E3']
shifts = ['Morning', 'Evening']
# Cost matrix
cost = {('E1','Morning'): 10, ('E1','Evening'): 12,
        ('E2','Morning'): 8, ('E2','Evening'): 9,
        ('E3','Morning'): 7, ('E3','Evening'): 11}
# Shift requirements
requirements = {'Morning': 2, 'Evening': 1}
# Model
model = pulp.LpProblem('Employee_scheduling', pulp.LpMini.)
# Decision variables
x = pulp.LpVariables.dicts('assign', (employee, shifts), cat
    = 'Binary')
# Objective
```

```
model += pulp.lpSum(cost[i,j] * x[i][j] for i in range
employee for j in shifts)
# Shift coverage constraints
for j in shifts:
    model += pulp.lpSum(x[i][j] for i in employee)
        >= requirements[j]
# One shift per employee
for i in employees:
    model += pulp.lpSum(x[i][j] for j in shifts) <=
# Solve
model.solve()
# Output
for i in employees:
    for j in shifts:
        if pulp.value(x[i][j]) == 1
            print(f'{i} works in {j}')
```

* **CONCLUSION :-**

Employee scheduling problem was solved optimally.

* ! END ! *

OUTPUT :-

Status : Optimal
E1 works in Morning Shift (cost : 10)
E2 works in Evening Shift (cost : 9)
E3 works in Morning Shift (cost : 7)
Total Maximum / Minimum cost : 26

EXPERIMENT NO. 01

* AIM :- Optimization of assignment problem ~~and Branch and Bound method.~~

* THEORY :- Assignment problem assign n workers to n task to minimize total cost. The Hungarian algorithm solves it in $O(n^3)$ time. Cost matrix C[i][j], objective minimize $\sum C[i][j]$.

* CODE :-

```
import numpy as np
from scipy.optimize import linear_sum_assignment
cost_matrix = np.array([[9,2,7,8],
                        [6,4,3,7],
                        [5,8,1,8],
                        [7,6,9,4]])
row_ind, col_ind = linear_sum_assignment(cost_matrix)
print('Optimal Assignments:')
for r, c in zip(row_ind, col_ind):
    print(f'Worker {r+1} -> Task {c+1} {cost: {cost_matrix[r][c]}}')
print('Total minimum cost:', cost_matrix[row_ind, col_ind].sum())
```

* CONCLUSION :-

Assignment problem is solved successfully.

* ! END ! *

* OUTPUT :-

Optimal assignments :

Worker 1 → Task 2 (cost : 2)

Worker 2 → Task 1 (cost : 6)

Worker 3 → Task 3 (cost : 1)

Worker 4 → Task 4 (cost : 4)

Total minimum cost is : 13.

```
if bound <= best value [0]:
    return # Prune

branch_and_bound (weights, values, capacity, index+1,
                  current_weight + weight [index],
                  current value + value [index])

branch_and_bound (weights, values, capacity, index+1,
                  current_weight, current_value)
weights = (2,3,4,5)
values = (3,4,5,6)
capacity = 8
branch_and_bound (weights, values, capacity, 0,0,0)
print ("\n max knapsack value (Branch & Bound):",
    best_value [0]).
```

* **CONCLUSION :-**

Branch and Bound algorithm is implemented successfully and optimally.

* ! END ! *

EXPERIMENT NO. 01

* **Aim :-** Implementing Branch and Bound method.

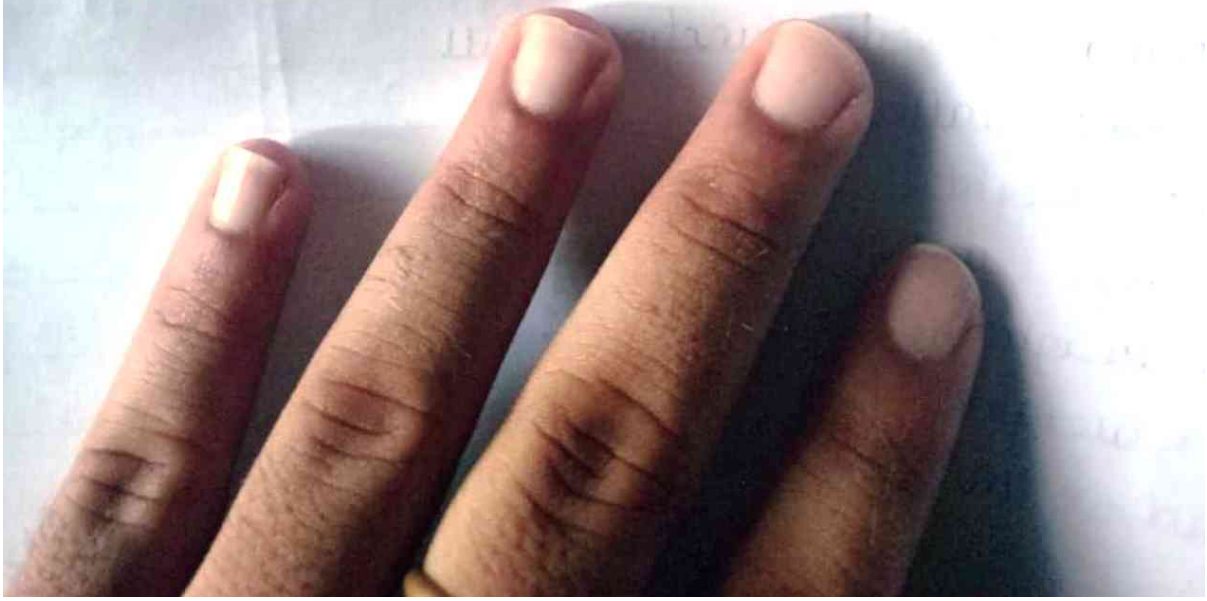
* **THEORY :-** It is general optimization technique for integer / combinatorial problems. It divide problem into subproblem by fixing variables. It guarantees optimal solution and is basis for most MIP (Mixed integer programming) solvers.

* **CODE :-**

```
best = value = [0]
def branch_and_bound(weight, value, capacity, index,
current_weight, current_value):
  if current_weight > capacity:
    return
  if current_value > best_value[0]:
    best_value[0] = current_value
  if index == len(weight):
    return
#Upper bound: greedy functional fill
bound = current_value
w = current_weight
for i in range(index, len(weights)):
  if w + weight[i] <= capacity:
    w += weights[i]
    bound += values[i]
  else:
    bound += (capacity-w) * values[i]/weights[i]
    break
```

* OUTPUT :-

```
Max Knapsack value
(Branch & Bound) : 10
```



Experiment 5

[1]
✓ 0s

```
import numpy as np
from scipy.optimize import linprog

# Coefficients of objective function
# Max -> convert to minimize
c = np.array([-3, -5])

# Constraint coefficients
A = np.array([
    [2, 3],
    [1, 1]
])

# Right-hand side
b = np.array([12, 6])

# Bounds for variables (x1, x2 >= 0)
x_bounds = (0, None)
y_bounds = (0, None)

# Solve
result = linprog(c, A_ub=A, b_ub=b,
                 bounds=(x_bounds, y_bounds))

print("Optimal value =", -result.fun)
print("x1, x2 =", result.x)

print("Status:", result.message)
```

```
Optimal value = 20.0
x1, x2 = [0. 4.]
Status: Optimization terminated successfully. (HiGHS Status 7: Optimal)
```

Experiment 6

Code:

```
import pulp
import numpy as np

# Distance matrix
dist = np.array([
    [0, 10, 15, 20],
    [10, 0, 35, 25],
    [15, 35, 0, 30],
    [20, 25, 30, 0]
])

n = len(dist)

# Create model
model = pulp.LpProblem("TSP", pulp.LpMinimize)

# Decision variables
x = pulp.LpVariable.dicts(
    'x',
    (range(n), range(n)),
    cat='Binary'
)

# MTZ variables
u = pulp.LpVariable.dicts(
    'u',
    range(n),
    lowBound=0,
    upBound=n-1,
    cat='Integer'
)

# Objective function
model += pulp.lpSum(
    dist[i][j] * x[i][j]
    for i in range(n)
    for j in range(n)
)

# No self-loop
for i in range(n):
```

```
    model += x[i][i] == 0

# Each city visited once (outgoing)
for i in range(n):
    model += pulp.lpSum(x[i][j] for j in range(n)) == 1

# Each city visited once (incoming)
for j in range(n):
    model += pulp.lpSum(x[i][j] for i in range(n)) == 1

# MTZ subtour elimination
for i in range(1, n):
    for j in range(1, n):
        if i != j:
            model += u[i] - u[j] + n * x[i][j] <= n - 1

# Solve
model.solve()

# Output
print("Status:", pulp.LpStatus[model.status])
print("Tour:")

total_cost = 0

for i in range(n):
    for j in range(n):
        if pulp.value(x[i][j]) == 1:
            print(f"{i} -> {j} (dist: {dist[i][j]})")
            total_cost += dist[i][j]

print("\nTotal tour cost =", total_cost)
```

Output:

```
Status: Optimal
Tour:
0 -> 2 (dist: 15)
1 -> 0 (dist: 10)
2 -> 3 (dist: 30)
3 -> 1 (dist: 25)

Total tour cost = 80
```

Aim: Write code for Ford–Fulkerson Algorithm in Python

```
import networkx as nx

G = nx.DiGraph()

# Add edges with capacities
G.add_edge('S', 'A', capacity=16)
G.add_edge('S', 'B', capacity=13)
G.add_edge('A', 'B', capacity=10)
G.add_edge('A', 'C', capacity=12)
G.add_edge('B', 'A', capacity=4)
G.add_edge('B', 'D', capacity=14)
G.add_edge('C', 'B', capacity=9)
G.add_edge('C', 'T', capacity=20)
G.add_edge('D', 'C', capacity=7)
G.add_edge('D', 'T', capacity=4)

flow_value, flow_dict = nx.maximum_flow(G, 'S', 'T')

print("Maximum Flow:", flow_value)
```

```
Maximum Flow: 23
```

Experiment 8

[1] ✓ 1s

```
import networkx as nx

G = nx.DiGraph()

# Add edges
G.add_edge('S', 'A', capacity=10, weight=2)
G.add_edge('S', 'B', capacity=5, weight=4)
G.add_edge('A', 'B', capacity=15, weight=1)
G.add_edge('A', 'T', capacity=10, weight=2)
G.add_edge('B', 'T', capacity=10, weight=3)

# Demand (flow requirement)
G.nodes['S']['demand'] = -15   # Supply
G.nodes['T']['demand'] = 15    # Demand

# Solve
flow_dict = nx.min_cost_flow(G)
cost = nx.cost_of_flow(G, flow_dict)

print("Flow on each edge:")

for u in flow_dict:
    for v in flow_dict[u]:
        if flow_dict[u][v] > 0:
            print(f"{u} -> {v}: {flow_dict[u][v]} units")

print("\nMinimum Cost:", cost)
```

```
Flow on each edge:
S -> A: 10 units
S -> B: 5 units
A -> T: 10 units
B -> T: 5 units

Minimum Cost: 75
```

Experiment 9

Code:

```
import pulp

# Employees and shifts
employees = ['E1', 'E2', 'E3']
shifts = ['Morning', 'Evening']

# Cost matrix
cost = {
    ('E1', 'Morning'): 10,
    ('E1', 'Evening'): 12,
    ('E2', 'Morning'): 8,
    ('E2', 'Evening'): 9,
    ('E3', 'Morning'): 7,
    ('E3', 'Evening'): 11
}

# Shift requirements
requirements = {
    'Morning': 2,
    'Evening': 1
}

# Model
model = pulp.LpProblem("Employee_Scheduling", pulp.LpMinimize)

# Decision variables
x = pulp.LpVariable.dicts(
    "Assign",
    (employees, shifts),
    cat='Binary'
)

# Objective function
model += pulp.lpSum(
    cost[(i, j)] * x[i][j]
    for i in employees
    for j in shifts
)

# Shift coverage constraints
for j in shifts:
    model += pulp.lpSum(x[i][j] for i in employees) >= requirements[j]
```

```
# One shift per employee
for i in employees:
    model += pulp.lpSum(x[i][j] for j in shifts) <= 1

# Solve
model.solve()

# Output
print("Status:", pulp.LpStatus[model.status])

total_cost = 0

for i in employees:
    for j in shifts:
        if pulp.value(x[i][j]) == 1:
            print(f"{i} works in {j} shift (Cost: {cost[(i,j)]})")
            total_cost += cost[(i, j)]

print("Total Minimum Cost:", total_cost)
```

Output:

```
Status: Optimal
E1 works in Morning shift (Cost: 10)
E2 works in Evening shift (Cost: 9)
E3 works in Morning shift (Cost: 7)
Total Minimum Cost: 26
```

Aim: Write a code for Optimization of Assignment Problem using Python Programming

```
from scipy.optimize import linear_sum_assignment
import numpy as np

# Cost matrix
cost_matrix = np.array([
    [9, 2, 7],
    [6, 4, 3],
    [5, 8, 1]
])

# Solve assignment problem
row_ind, col_ind = linear_sum_assignment(cost_matrix)

# Display assignment
print("Optimal Assignment:")

total_cost = 0
for i in range(len(row_ind)):
    print(f"Worker {row_ind[i]+1} assigned to Job {col_ind[i]+1}")
    total_cost += cost_matrix[row_ind[i], col_ind[i]]

print("Minimum Total Cost:", total_cost)
```

```
Optimal Assignment:
Worker 1 assigned to Job 2
Worker 2 assigned to Job 1
Worker 3 assigned to Job 3
Minimum Total Cost: 9
```

Aim: Write code for Implementation of Branch and Bound Method using Python

Code:

```
from scipy.optimize import linprog
import math

# Objective function coefficients
# Maximize Z = 3x + 2y
c = [-3, -2]  # Negative for maximization

# Constraints
A = [[2, 1],
     [1, 2]]

b = [6, 6]

# Solve LP relaxation
result = linprog(c, A_ub=A, b_ub=b, bounds=(0, None))

x, y = result.x

print("LP Relaxation Solution:")
print("x =", round(x, 2))
print("y =", round(y, 2))
print("Maximum Value =", round(-result.fun, 2))

# Branch and Bound check
if x.is_integer() and y.is_integer():
    print("\nInteger solution found.")
else:
    print("\nApplying Branch and Bound...")

    x_int = math.floor(x)
    y_int = math.floor(y)

    print("Approx Integer Solution:")
    print("x =", x_int)
    print("y =", y_int)

    max_value = 3 * x_int + 2 * y_int
    print("Maximum Integer Value =", max_value)
```

Output:

```
LP Relaxation Solution:
x = 2.0
y = 2.0
Maximum Value = 10.0

Integer solution found.
```